

DEV 140 · DISCUSSION 4 · RESPONSIVE LAYOUTS

Layouts for Every Screen

Grid, Flexbox, and media queries

RECAP · WHERE WE ARE

Three steps to a real website

1 DISCUSSION 2
Build it

`index.html`

The content: headings, paragraphs, images, and links.

2 DISCUSSION 3
Style it

`style.css`

The look: colors, fonts, sizes, and spacing.

3 TODAY
Make it fit

`grid · flexbox · @media`

The layout: works on a phone, a tablet, and a desktop.

WHY DOES IT NEED TO FIT?

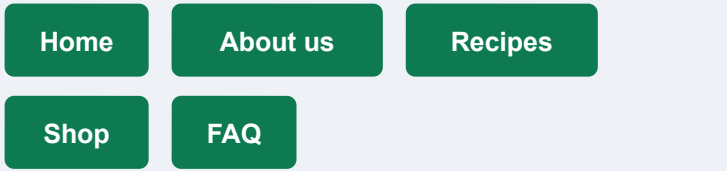
Phones come first
More than half of all web visits happen on a phone.

No pinching or scrolling
Visitors leave if they have to pinch and drag to read.

One page, every screen
You write the page once. It has to rearrange itself to fit.

Flexbox works in one direction, Grid in two

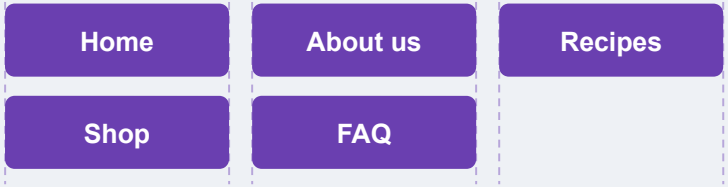
FLEXBOX · ONE DIRECTION




```
nav ul {
  display: flex;
  flex-wrap: wrap;
}
```

Items sit side by side, each as wide as its content. Extra items wrap to a new line, but they don't line up with the line above.

GRID · ROWS AND COLUMNS




```
.card-grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
}
```

You choose the columns, and every item lines up with the ones above and below it. `1fr` = one equal share of the space.

Which one should I use?

Is it one line of things?

Use Flexbox

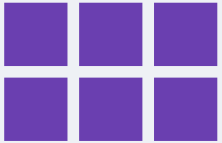
- Nav links
- A row of buttons
- A logo next to a menu



Should things line up in columns?

Use Grid

- Cards (today's example)
- A photo gallery
- A list of products or people

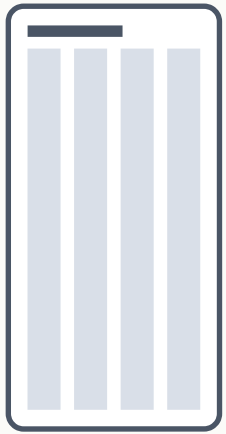


Both go on the holder
Put display on the box that holds the items, not on each item.

Use both on one page
Grid for the cards, Flexbox for the nav. They don't fight.

Not sure?
Either one earns full credit on the homework. Pick one and try it.

One layout can't fit every screen



✗ **Phone: 4 columns**
Squished. Too narrow to read.



✗ **Desktop: 1 column**
Stretched out. Lots of wasted space.

THE FIX

Media queries

Write different CSS for different screen widths. Same HTML, different layout.



How a media query works

```
@media (min-width: 768px) {
  body {
    background-color: #e3eefa;
  }
  .card-grid {
    grid-template-columns: 1fr 1fr;
  }
}
```

1. **`@media`**
   Starts a media query: a question for the browser.

2. **`(min-width: 768px)`**
   The question: is the screen at least 768px wide?

3. **`{ ... }`**
   YES: use the rules inside. NO: skip them.

EXAMPLE: A SCREEN THAT IS 900px WIDE

Normal rules
✓ Always used

→

768px or wider?
✓ Yes: use them

→

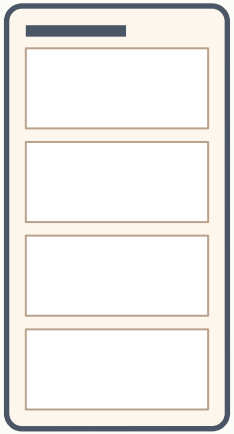
1200px or wider?
✗ No: skip them

→

Result
Light blue, 2 columns

MEDIA QUERIES · 3 OF 3

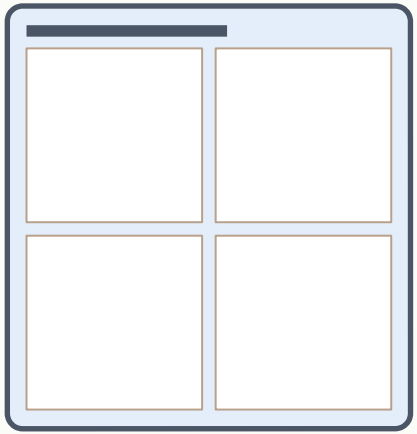
Today's page: one page, three layouts



PHONE

`normal CSS only`

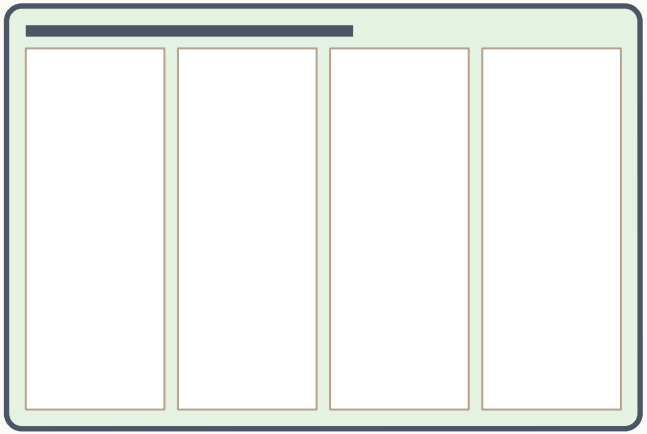
1 column · cream



TABLET

`@media (min-width: 768px)`

2 columns · light blue



DESKTOP

`@media (min-width: 1200px)`

4 columns · light green

Test it: F12, then Ctrl+Shift+M (Cmd+Shift+M on Mac). Type 320, 768, and 1200 into the width box. Drag slowly across 768 and watch the color flip.

BEFORE YOU SUBMIT

Check these four things on your page

1. **Grid or Flexbox**
   On a section with 2 or more items, making columns or a row.

2. **A media query**
   The layout visibly changes when you cross the width.

3. **Phone · 320px**
   One column, readable, no sideways scrolling. Images get max-width: 100%.

4. **Tablet · 768px**
   Looks different from both the phone and the desktop.

Then: commit, push, and submit your GitHub repo URL.